

计算机系统结构

第3章 流水线技术

目录

- 3.1 [流水线的基本概念](#)
- 3.2 [流水线的性能指标](#)
- 3.3 [非线性流水线的调度](#)
- 3.4 [流水线的相关与冲突](#)
- 3.5 [流水线的实现](#)

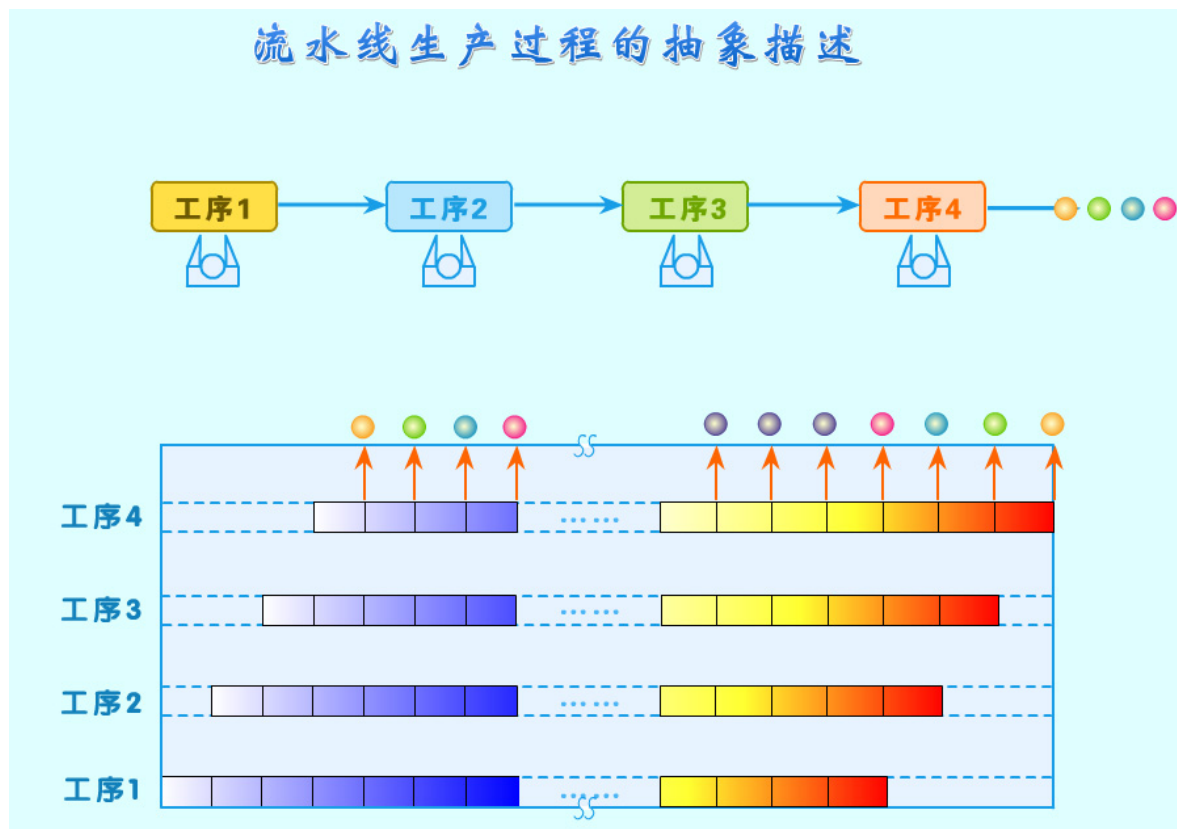
3.1 基本概念

3.1 流水线的基本概念

3.1.1 什么是流水线

1. 工业生产流水线

1769年英国陶瓷工厂把一个人从头到尾完成的制陶工艺分成几十道专门工序。
亨利·福特在福特公司发明流水线装配工艺，用于汽车生产。



3.1 流水线的基本概念

2. 流水线技术（Pipelining）

- 把一个重复的过程分解为若干个子过程，每个子过程由专门的功能部件来实现。
- 把多个处理过程在时间上错开，依次通过各功能段，这样，每个子过程就可以与其它子过程并行进行。

3. 流水线中的每个子过程及其功能部件称为流水线的级或段（Stage），段与段相互连接形成流水线。流水线的段数称为流水线的深度（Pipeline Depth）。

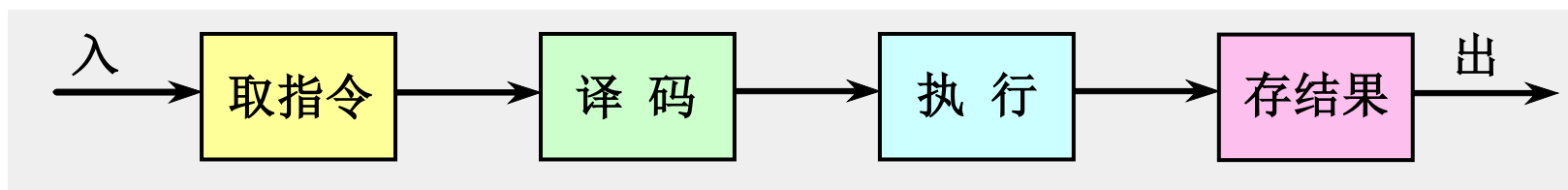
3.1 流水线的基本概念

4. 指令流水线

- 把指令的解释过程分解为译码和执行两个子过程，并让这两个子过程分别用独立的译码部件和执行部件来实现。

理想情况：速度提高一倍

- 4段指令流水线



3.1 流水线的基本概念

5. 浮点加法流水线

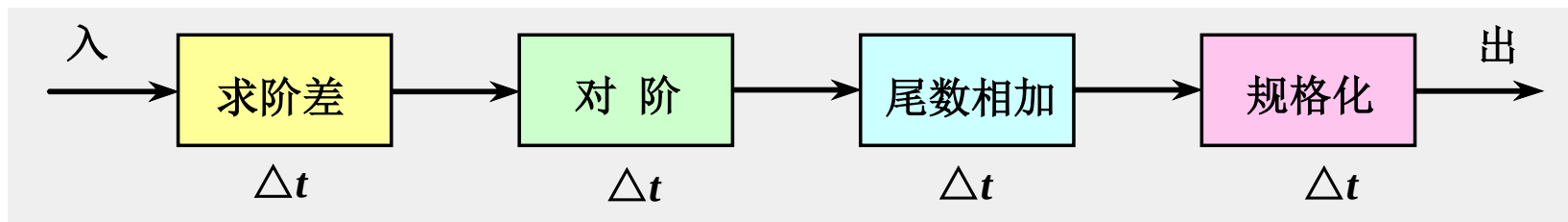
- 把流水线技术应用于运算的执行过程，就形成了运算操作流水线，也称为部件级流水线。
- 把浮点加法的全过程分解为求阶差、对阶、尾数相加、规格化四个子过程。

例： $x = .1010 \times 2^{10}$, $y = .1100 \times 2^{01}$

$x + y = .1010 \times 2^{10} + .0110 \times 2^{10} = (.1010 + .0110) \times 2^{10}$

尾数求和： $(.1010 + .0110)$

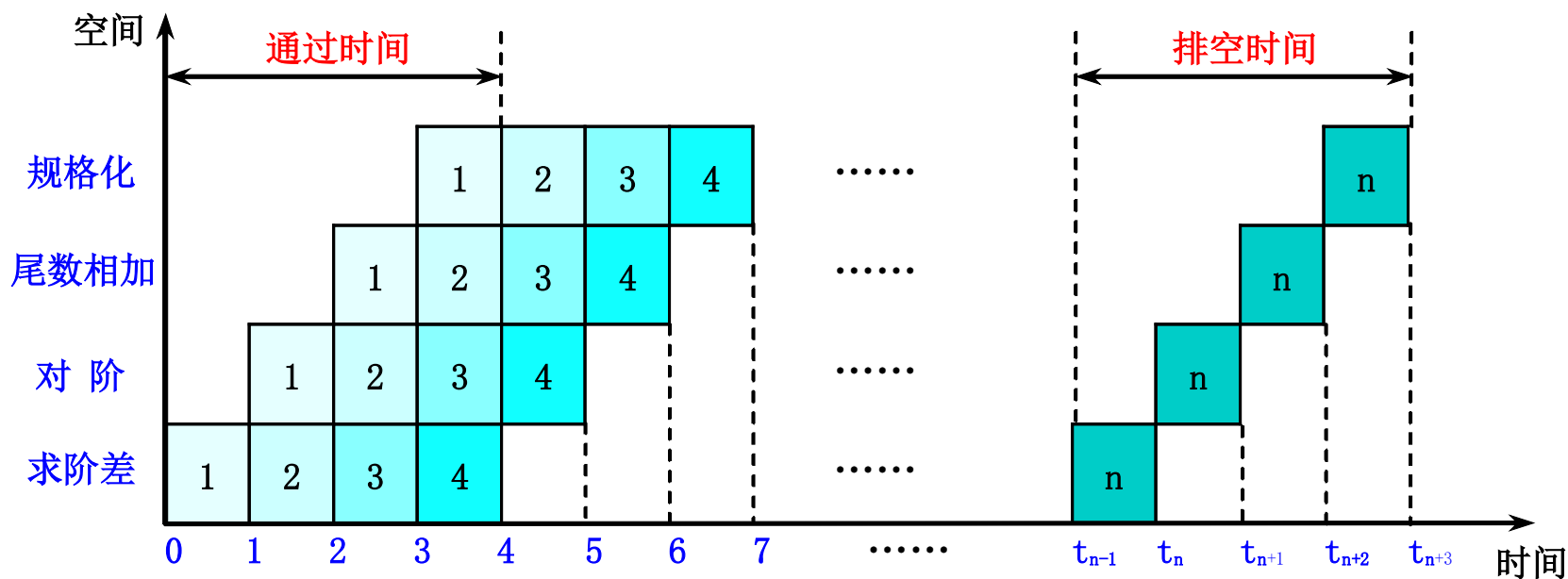
理想情况：速度提高3倍



3.1 流水线的基本概念

6. 时一空图

- 时一空图从时间和空间两个方面描述了流水线的工作过程。
时一空图中，横坐标代表时间，纵坐标代表流水线的各个段。
- **通过时间**：第一个任务从进入流水线到流出结果的时间段
- **排空时间**：最后一个



浮点加法流水线的时空图

3.1 流水线的基本概念

7. 流水技术的特点

- 流水线把一个处理过程分解为若干个子过程（段），每个子过程由一个专门的功能部件来实现。
- 流水线中各段的时间应尽可能相等，否则将引起流水线堵塞、断流。

时间最长的段将成为流水线的瓶颈。

- 流水线每一个段的后面都要有一个缓冲寄存器（锁存器），称为流水寄存器。
 - ▣ 作用：在相邻的两段之间传送数据，以保证提供后面要用到的信息，并把各段的处理工作相互隔离。
- 流水技术适合于大量重复的时序过程，只有在输入端不断地提供任务，才能充分发挥流水线的效率。

3.1 流水线的基本概念

3.1.2 流水线的分类

从不同的角度和观点，把流水线分成多种不同的种类。

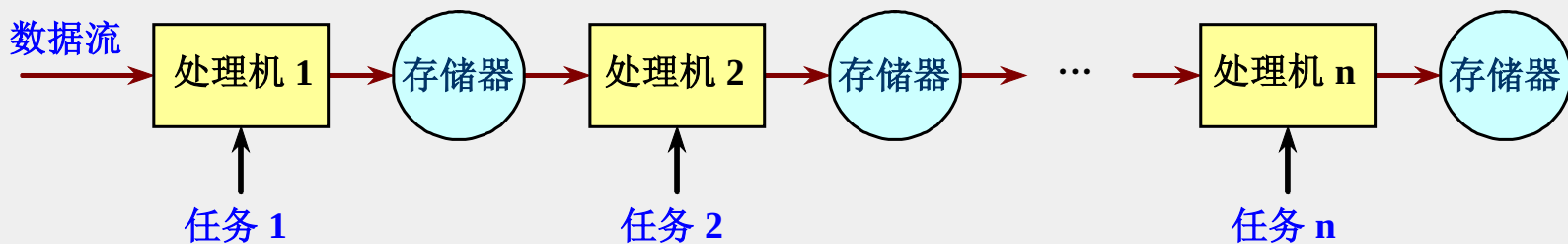
1. 部件级、处理机级及系统级流水线

（按照流水技术用于计算机系统的等级不同）

- **部件级流水线**（运算操作流水线）：把处理机中的部件分段，再把这些分段相互连接起来，使得各种类型的运算操作能够按流水方式进行。
- **处理机级流水线**（指令流水线）：把指令的执行过程按照流水方式处理。把一条指令的执行过程分解为若干个子过程，每个子过程在独立的功能部件中执行。

3.1 流水线的基本概念

- **系统级流水线**（宏流水线）：把多台处理机串行连接起来，对同一数据流进行处理，每个处理机完成整个任务中的一部分。



处理机间流水线

将多台处理机串行连接起来，对同一数据流进行处理，每个处理机完成整个任务中的一部分。



(PB级，甚至是EB级)



计算框架Map Reduce
一条简单的流水线



分布式架构

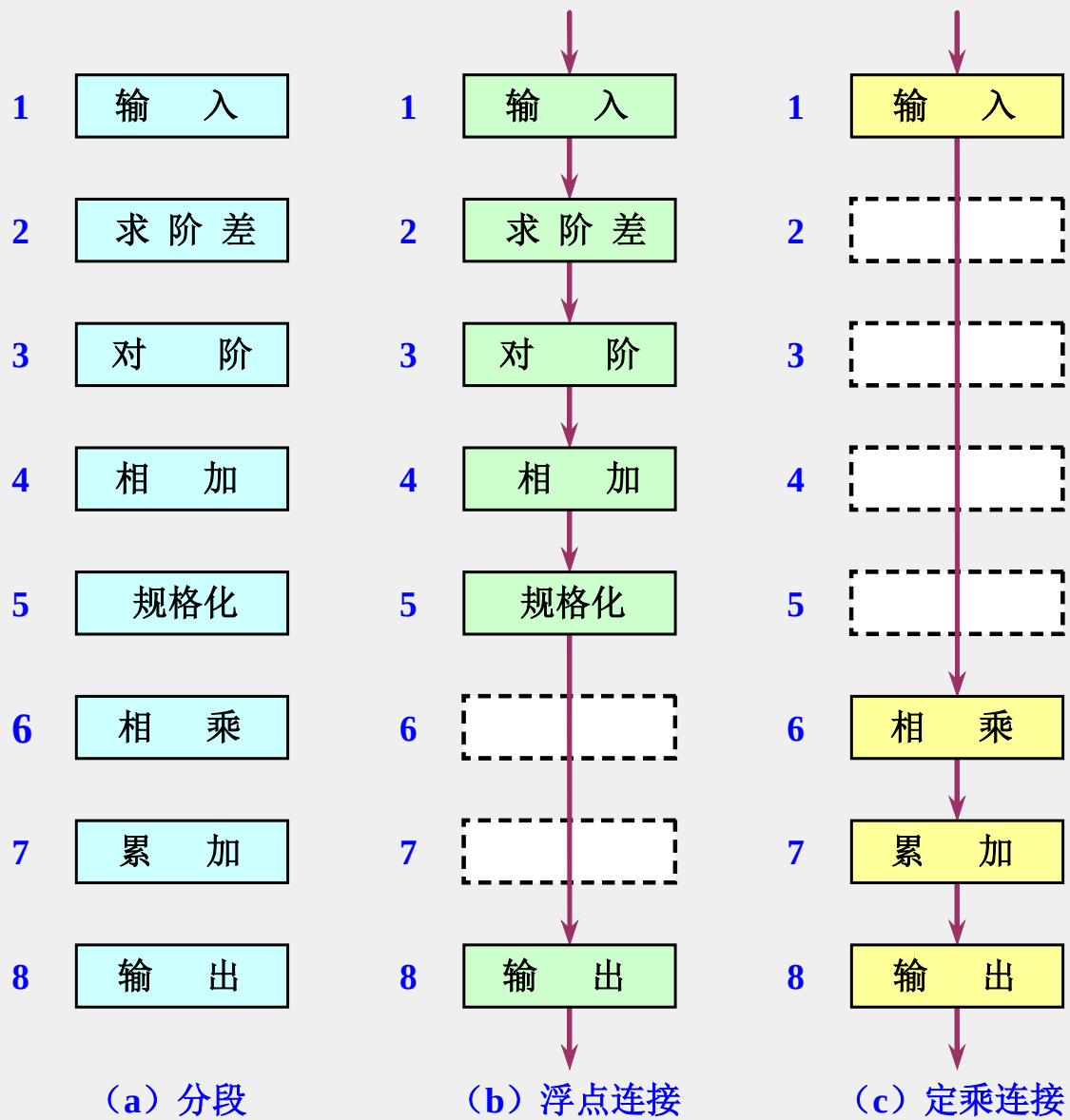
3.1 流水线的基本概念

2. 单功能流水线与多功能流水线

（按照流水线所完成的功能来分类）

- **单功能流水线：**只能完成一种固定功能的流水线。
- **多功能流水线：**流水线的各段可以进行不同的连接，以实现不同的功能。

例： TI公司ASC处理机的多功能流水线
实现浮点加减法+定点乘法运算



TI公司ASC处理机的多功能流水线

3.1 流水线的基本概念

3. 静态流水线与动态流水线

（多功能流水线按照同一时间内各段之间的连接方式分类）

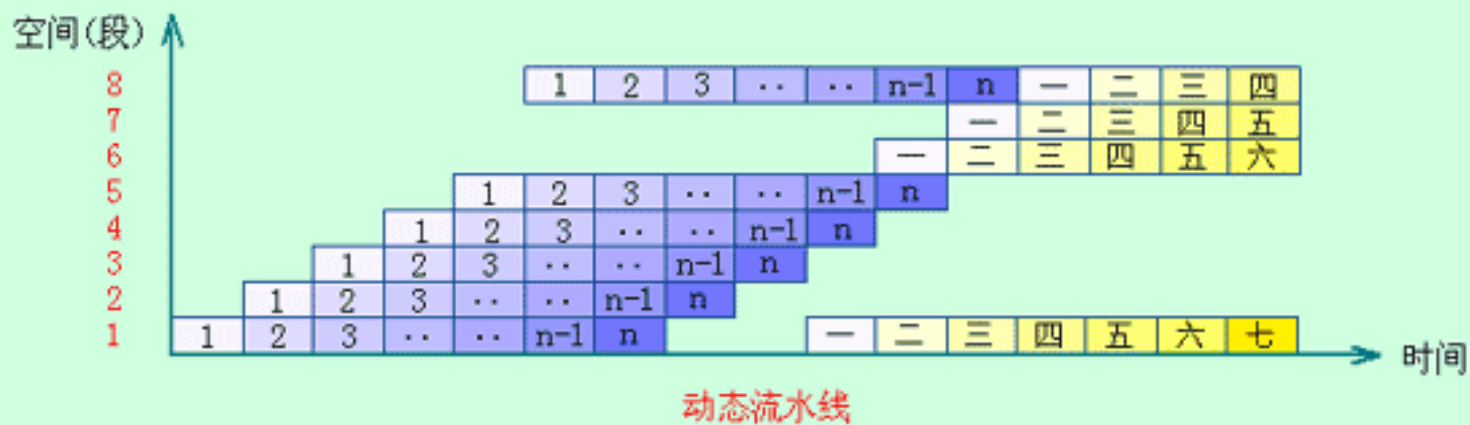
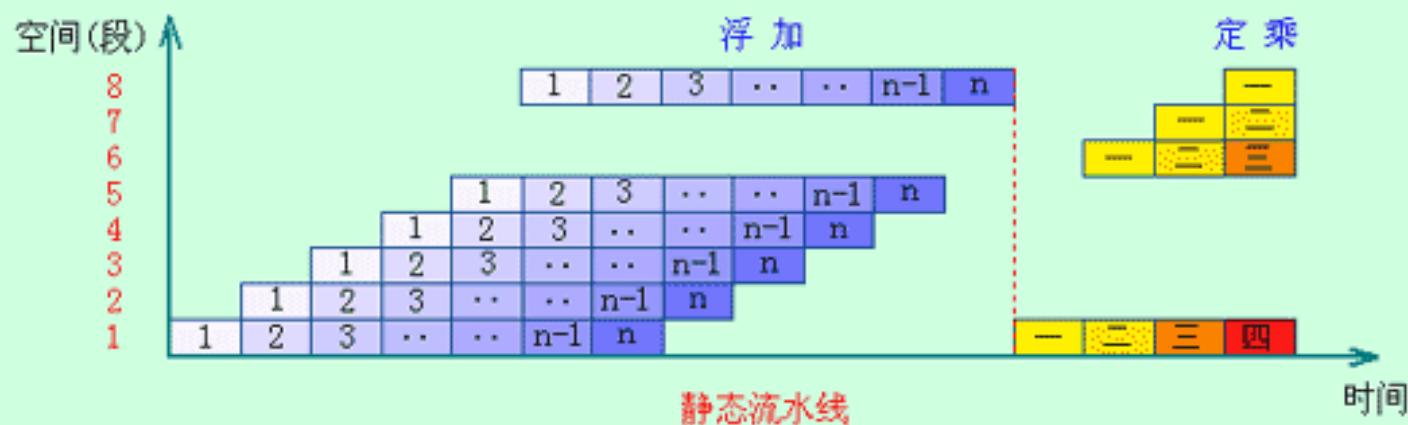
- **静态流水线（Static Pipeline）**：在同一时间内，多功能流水线中的各段只能按同一种功能的连接方式工作。流水线要切换到另一种功能时，必须等前面的任务都流出流水线之后，才能改变连接。
 - 对于静态流水线来说，只有当输入的是一串相同的运算任务时，流水的效率才能得到充分的发挥。

3.1 流水线的基本概念

- **动态流水线 (Dynamic Pipeline)**：在同一时间内，多功能流水线中的各段可以按照不同的方式连接，同时执行多种功能。当然，多功能流水线中的任何一个功能段只能参加到一种连接中。
 - **优点**
灵活，能够提高流水线各段的使用率，从而提高处理速度。
 - **缺点**
控制复杂。
- 静、动态流水线时空图的对比

静、动态流水线的时空图

假设该流水线要先做几个浮点加法，然后再做一批定点乘法。



3.1 流水线的基本概念

4. 线性流水线与非线性流水线

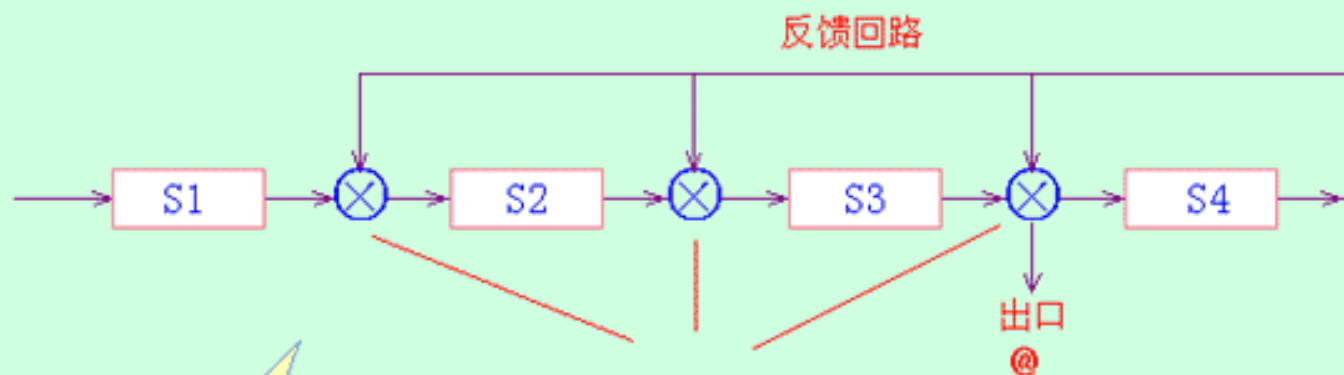
（按照流水线中是否有反馈回路来进行分类）

- **线性流水线（Linear Pipeline）**：流水线的各段串行连接，没有反馈回路。数据通过流水线中的各段时，每一个段最多只流过一次。
- **非线性流水线（Nonlinear Pipeline）**：流水线中除了有串行的连接外，还有反馈回路。
- 非线性流水线的**调度问题**
 - 确定什么时候向流水线引进新的任务，才能使该任务不会与先前进入流水线的任务发生冲突——争用流水段。

3.1 流水线的基本概念

非线性流水线

(举例)



虽然流水线仅由四段构成，
但有些段可能要重复通过。

串行连接

例如任务 @：

→ S1 → S2 → S3 → S4 → S2 → S3 → S4 → S3 →

3.1 流水线的基本概念

5. 顺序流水线与乱序流水线

（根据任务流入和流出的顺序是否相同来进行分类）

- **顺序流水线（In-order Pipeline）**：流水线输出端任务流出的顺序与输入端任务流入的顺序完全相同。每一个任务在流水线的各段中是一个跟着一个顺序流动的。
- **乱序流水线（Out-of-order Pipeline）**：流水线输出端任务流出的顺序与输入端任务流入的顺序可以不同，允许后进入流水线的任务先完成（从输出端流出）。

也称为无序流水线、错序流水线、异步流水线

3.1 流水线的基本概念

6. 标量处理机与向量流水处理机

- 把指令执行部件中采用了流水线的处理机称为**流水线处理机**。
- **标量处理机**：处理机不具有向量数据表示和向量指令，仅对标量数据进行流水处理。
- **向量流水处理机**：具有向量数据表示和向量指令的处理机。

向量数据表示和流水技术的结合。

3.2 性能指标

3.2 流水线的性能指标

3.2.1 吞吐率

吞吐率 (Throughput)：在单位时间内流水线所完成的任务数量或输出结果的数量。

$$TP = \frac{n}{T_k}$$

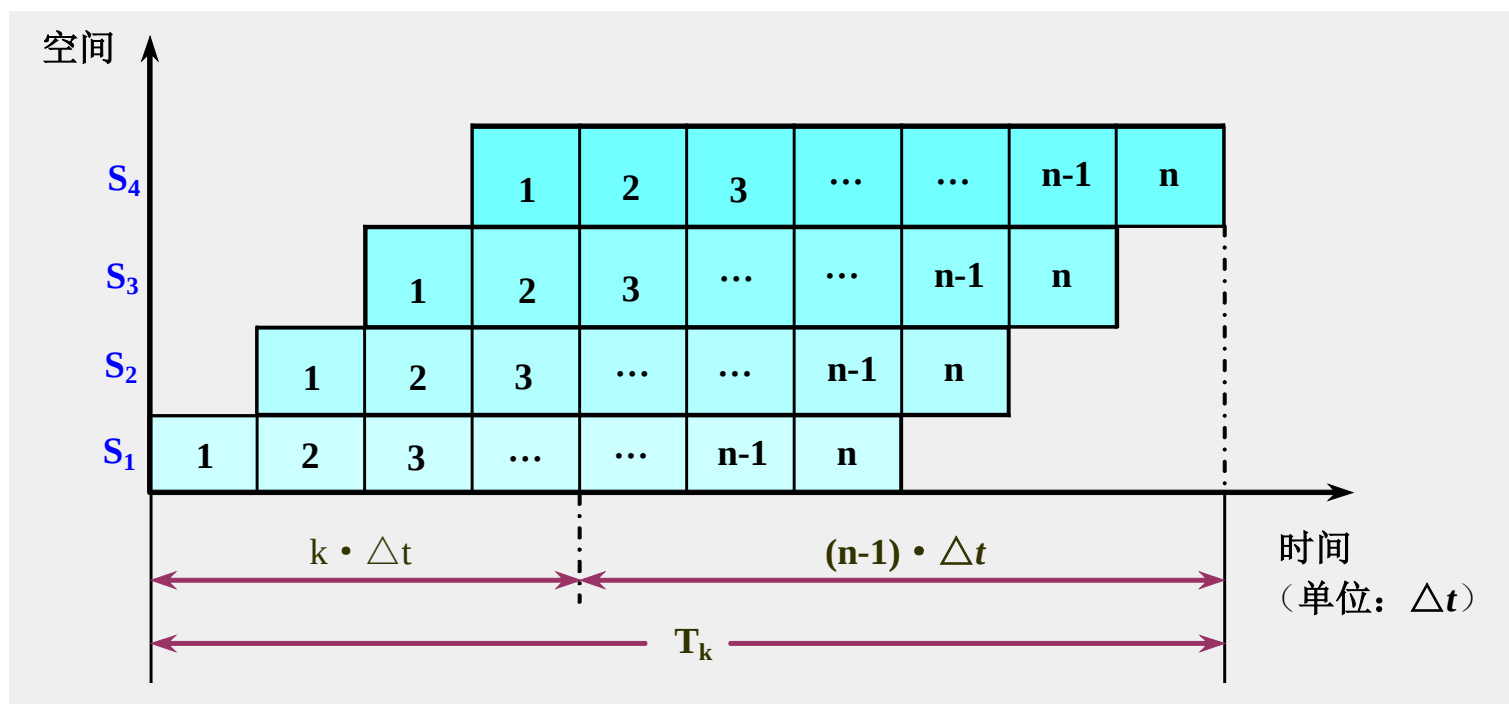
n ：任务数

T_k ：处理完成 n 个任务所用的时间

3.2 流水线的性能指标

1. 各段时间均相等的流水线

➤ 各段时间均相等的流水线时空图



周期时间 (Cycle time) : 完成一个流水段所用的时间。

3.2 流水线的性能指标

- 流水线完成 n 个连续任务所需要的总时间为：
(假设一条 k 段线性流水线)

$$T_k = k \Delta t + (n - 1) \Delta t = (k + n - 1) \Delta t$$

- 流水线的实际吞吐率

$$TP = \frac{n}{(k + n - 1) \Delta t}$$

- 最大吞吐率

$$TP_{\max} = \lim_{n \rightarrow \infty} \frac{n}{(k + n - 1) \Delta t} = \frac{1}{\Delta t}$$

3.2 流水线的性能指标

➤ 最大吞吐率与实际吞吐率的关系

$$TP = \frac{n}{k + n - 1} TP_{\max}$$

- ❑ 流水线的实际吞吐率小于最大吞吐率，它除了与每个段的时间有关外，还与流水线的段数 k 以及输入到流水线中的任务数 n 等有关。
- ❑ 只有当 $n \gg k$ 时，才有 $TP \approx TP_{\max}$ 。

3.2 流水线的性能指标

2. 各段时间不完全相等的流水线

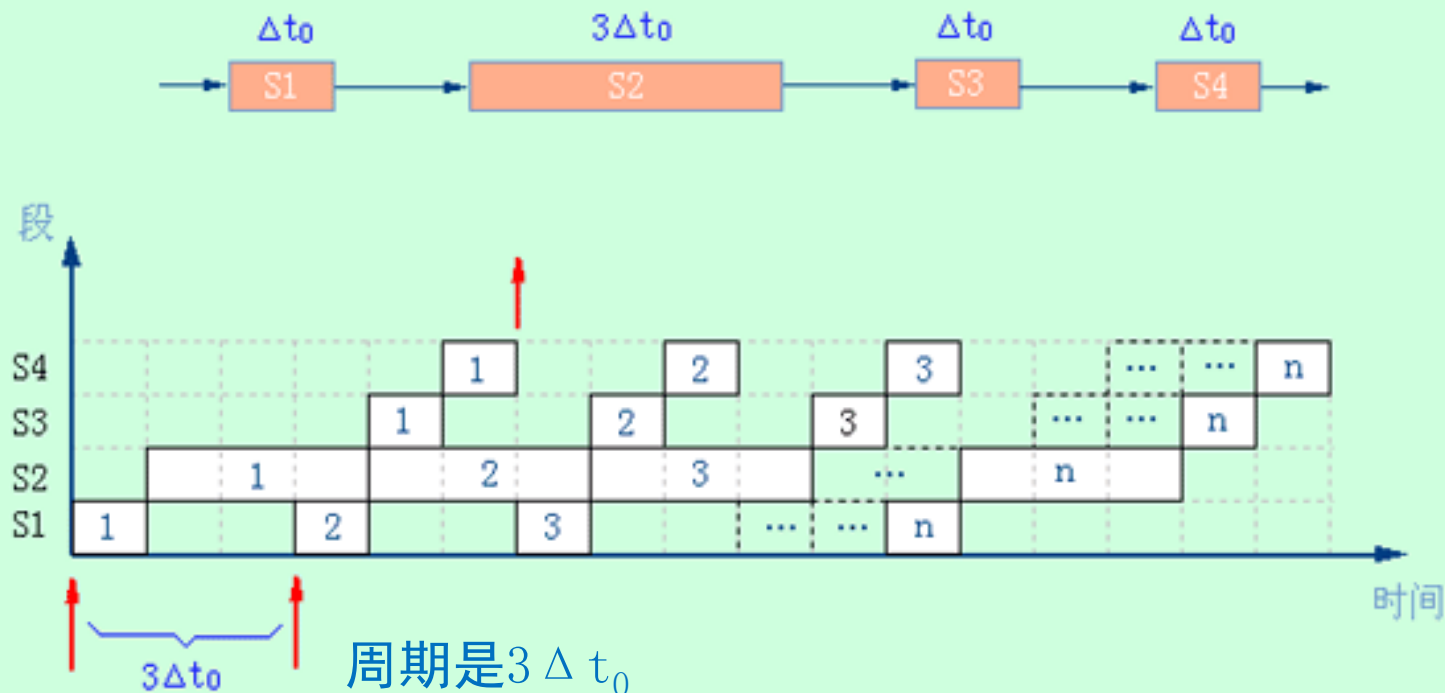
例1 一条4段的流水线

流水线中时间最长的段称为流水线的**瓶颈段**。

流水线的时-空图
(各段时间不等)

S1, S3, S4各段的时间: Δt

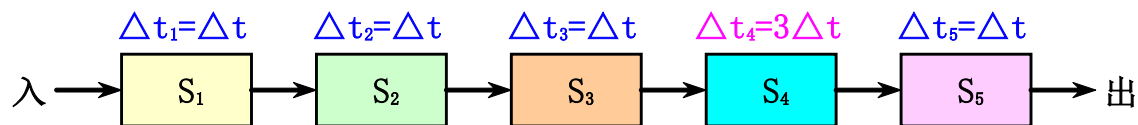
S2的时间: $3\Delta t$ (瓶颈段)



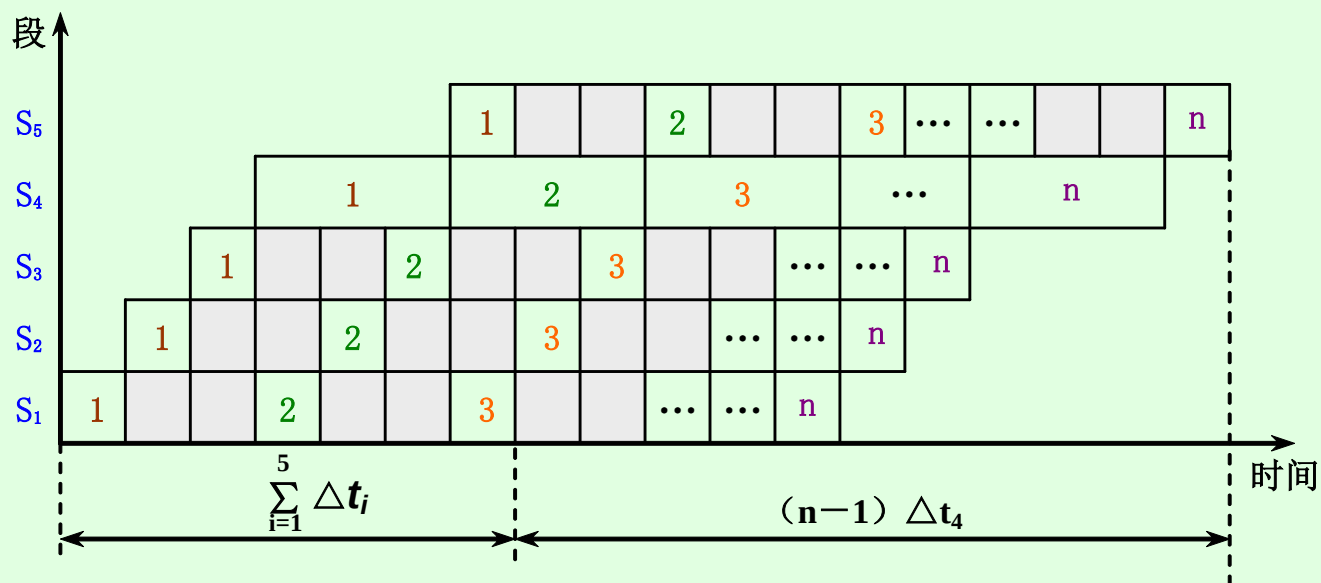
3.2 流水线的性能指标

例2：一条5段的流水线

- S_1, S_2, S_3, S_5 各段的时间: Δt
- S_4 的时间: $3\Delta t$ (瓶颈段)



(a) 流水线



(b) 时空图

3.2 流水线的性能指标

- 各段时间不等的流水线的**实际吞吐率**为：

（ Δt_i 为第 i 段的时间，共有 k 个段 ）

$$TP = \frac{n}{\sum_{i=1}^k \Delta t_i + (n-1) \max(\Delta t_1, \Delta t_2, \dots, \Delta t_k)}$$

- 流水线的**最大吞吐率**为：

$$TP_{\max} = \frac{1}{\max(\Delta t_1, \Delta t_2, \dots, \Delta t_k)}$$

对前面例2中的5段流水线

最大吞吐率为：

$$TP_{\max} = \frac{1}{3\Delta t}$$

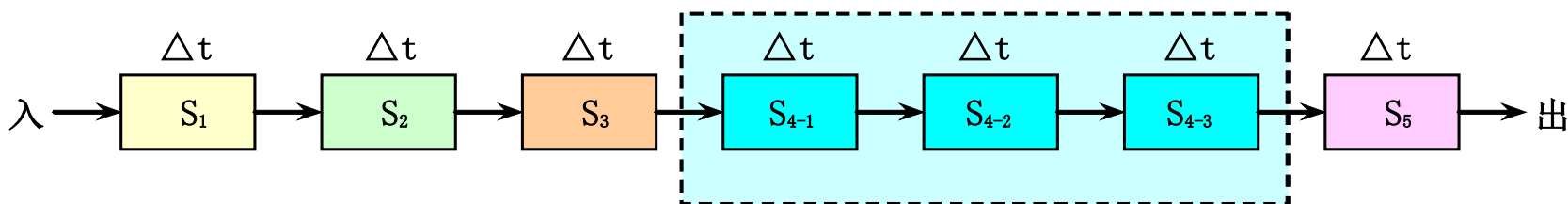
3.2 流水线的性能指标

3. 解决流水线瓶颈问题的常用方法

➤ 细分瓶颈段

例如：对前面的5段流水线

把瓶颈段 S_4 细分为3个子流水线段： S_{4-1} ， S_{4-2} ， S_{4-3}



改进后的流水线的吞吐率：

$$TP_{\max} = \frac{1}{\Delta t}$$

3.2 流水线的性能指标

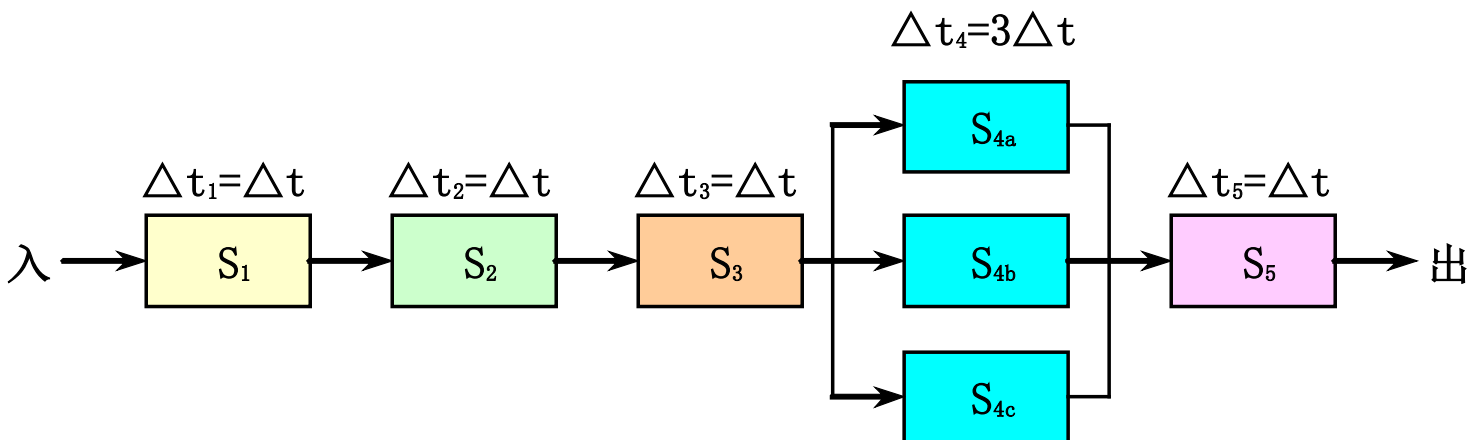
➤ 重复设置瓶颈段

- 举例：

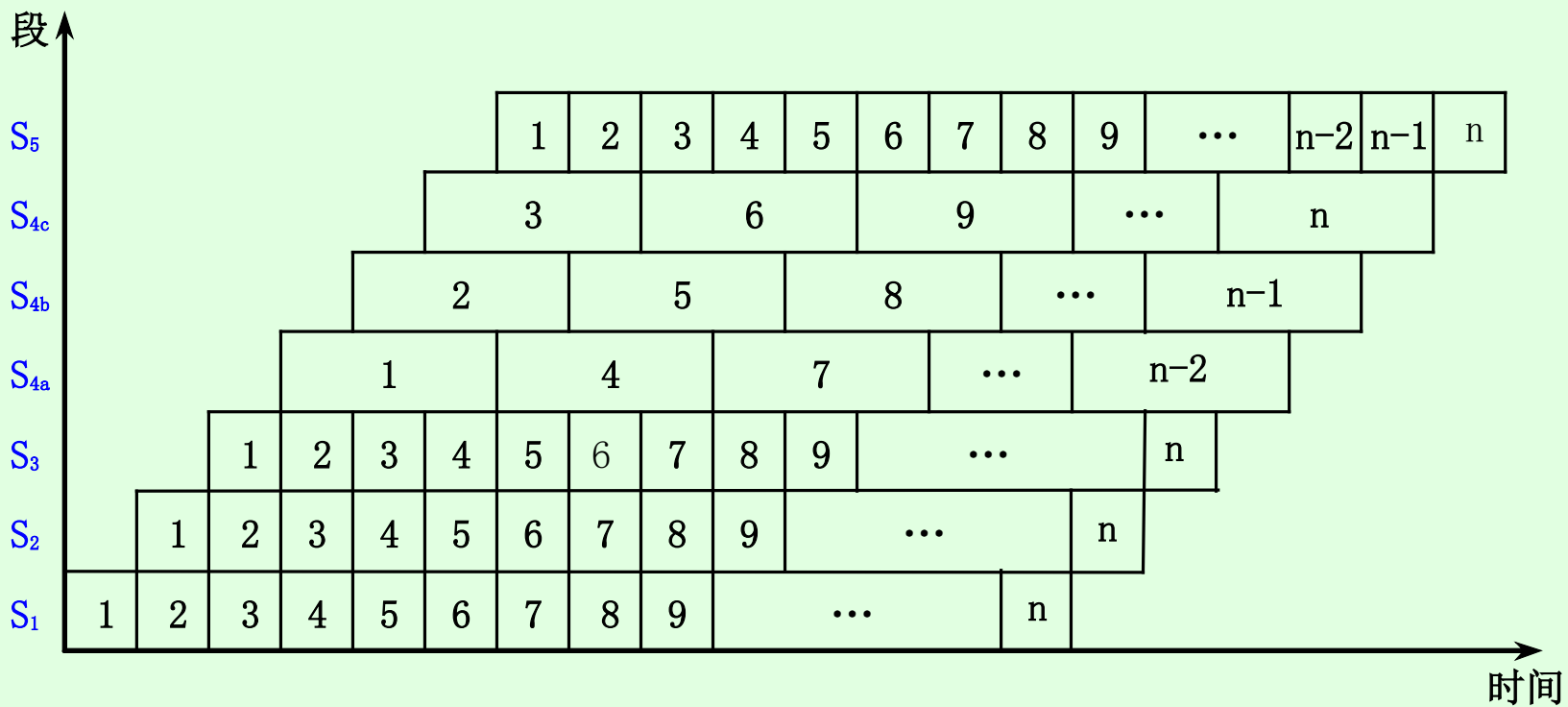
- **缺点：** 控制逻辑比较复杂，所需的硬件增加了。

例如： 对前面的5段流水线

重复设置瓶颈段 S_4 ： S_{4a} ， S_{4b} ， S_{4c}



3.2 流水线的性能指标



重复设置瓶颈段后的时空图

3.2 流水线的性能指标

3.2.2 流水线的加速比

加速比：完成同样一批任务，不使用流水线所用的时间与使用流水线所用的时间之比。

假设：不使用流水线（即顺序执行）所用的时间为 T_s ，使用流水线后所用的时间为 T_k ，则该流水线的加速比为：

$$S = \frac{T_s}{T_k}$$

3.2 流水线的性能指标

1. 流水线各段时间相等（都是 Δt ）

- 一条 k 段流水线完成 n 个连续任务

所需要的时间为： $T_k = (k+n-1)\Delta t$

- 顺序执行 n 个任务

所需要的时间： $T_s = nk\Delta t$

- 流水线的实际加速比为：

$$S = \frac{nk}{k+n-1}$$

- 最大加速比

$$S_{\max} = \lim_{n \rightarrow \infty} \frac{nk}{k+n-1} = k$$

当 $n \gg k$ 时， $S \approx k$

思考：流水线的段数愈多愈好？

3.2 流水线的性能指标

2. 流水线的各段时间不完全相等时

- 一条 k 段流水线完成 n 个连续任务的实际加速比为：

$$S = \frac{n \sum_{i=1}^k \Delta t_i}{\sum_{i=1}^k \Delta t_i + (n-1) \max(\Delta t_1, \Delta t_2, \dots, \Delta t_k)}$$

3.2 流水线的性能指标

3.2.3 流水线的效率

流水线的效率：流水线中的设备实际使用时间与整个运行时间的比值，即流水线设备的利用率。

由于流水线有通过时间和排空时间，所以在连续完成 n 个任务的时间内，各段并不是满负荷地工作。

1. 各段时间相等

➤ 各段的效率 e_i 相同

$$e_1 = e_2 = \cdots = e_k = \frac{n\Delta t}{T_k} = \frac{n}{k + n - 1}$$

3.2 流水线的性能指标

- 整条流水线的效率为：

$$E = \frac{e_1 + e_2 + \dots + e_k}{k} = \frac{ke_1}{k} = \frac{kn\Delta t}{kT_k}$$

- 可以写成：

$$E = \frac{n}{k + n - 1}$$

- 最高效率为：

$$E_{\max} = \lim_{n \rightarrow \infty} \frac{n}{k + n - 1} = 1$$

当 $n \gg k$ 时， $E \approx 1$ 。

- 当流水线各段时间相等时，流水线的效率与吞吐率成正比。

$$E = TP \cdot \Delta t$$

3.2 流水线的性能指标

- 流水线的效率是流水线的实际加速比 S 与它的最大加速比 k 的比值。

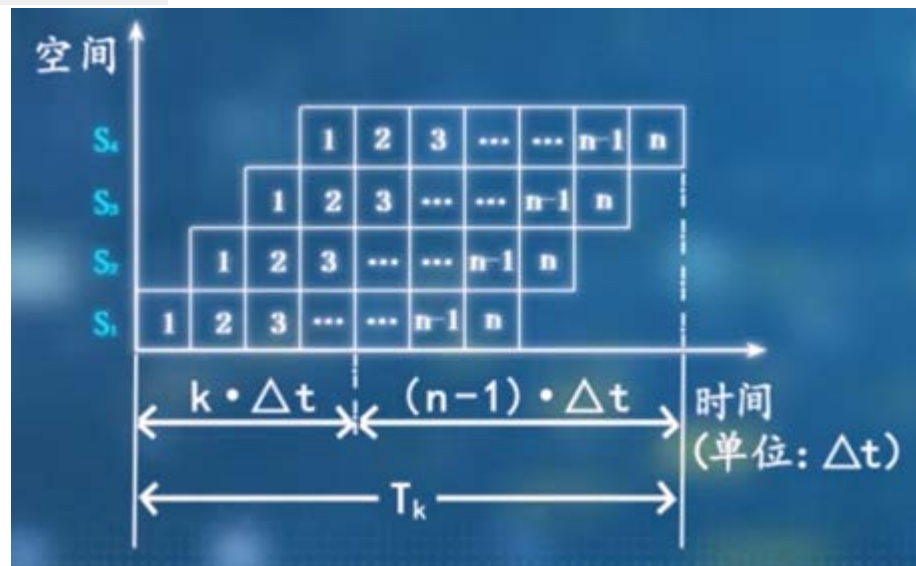
$$E = \frac{S}{k}$$

当 $E=1$ 时， $S=k$ ，实际加速比达到最大。

3.2 流水线的性能指标

3. 从时空图上看，效率就是 n 个任务占用的时空面积和 k 个段总的时空面积之比。

$$E = \frac{n \text{ 个任务实际占用的时空区}}{k \text{ 个段总的时空区}}$$



当各段时间不相等时:

$$E = \frac{n \cdot \sum_{i=1}^k \Delta t_i}{k \left[\sum_{i=1}^k \Delta t_i + (n-1) \cdot \max(\Delta t_1, \Delta t_2, \dots, \Delta t_k) \right]}$$

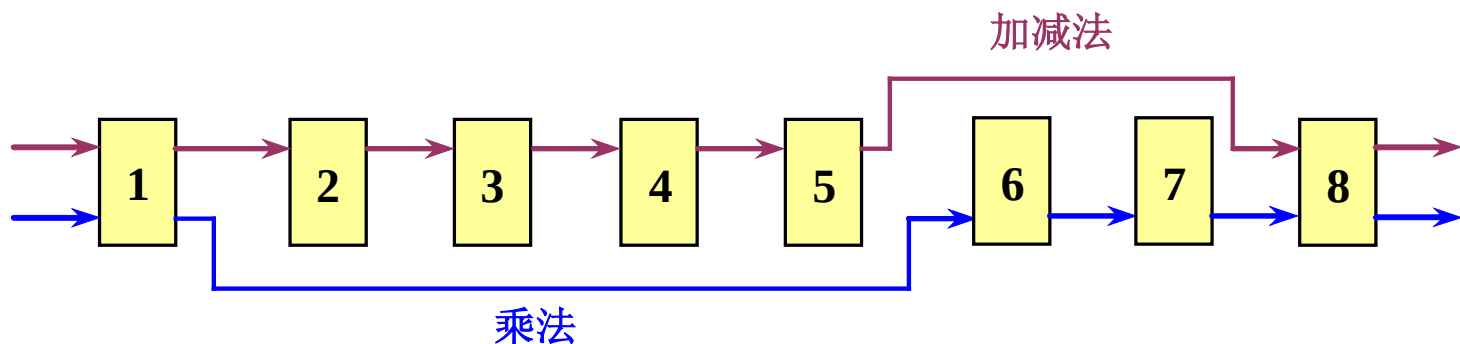
3.2 流水线的性能指标

3.2.4 流水线的性能分析举例

例3.1 设在下图所示的静态流水线上计算：

$$\prod_{i=1}^4 (A_i + B_i)$$

流水线的输出可以直接返回输入端或暂存于相应的流水寄存器中，试计算其吞吐率、加速比和效率。



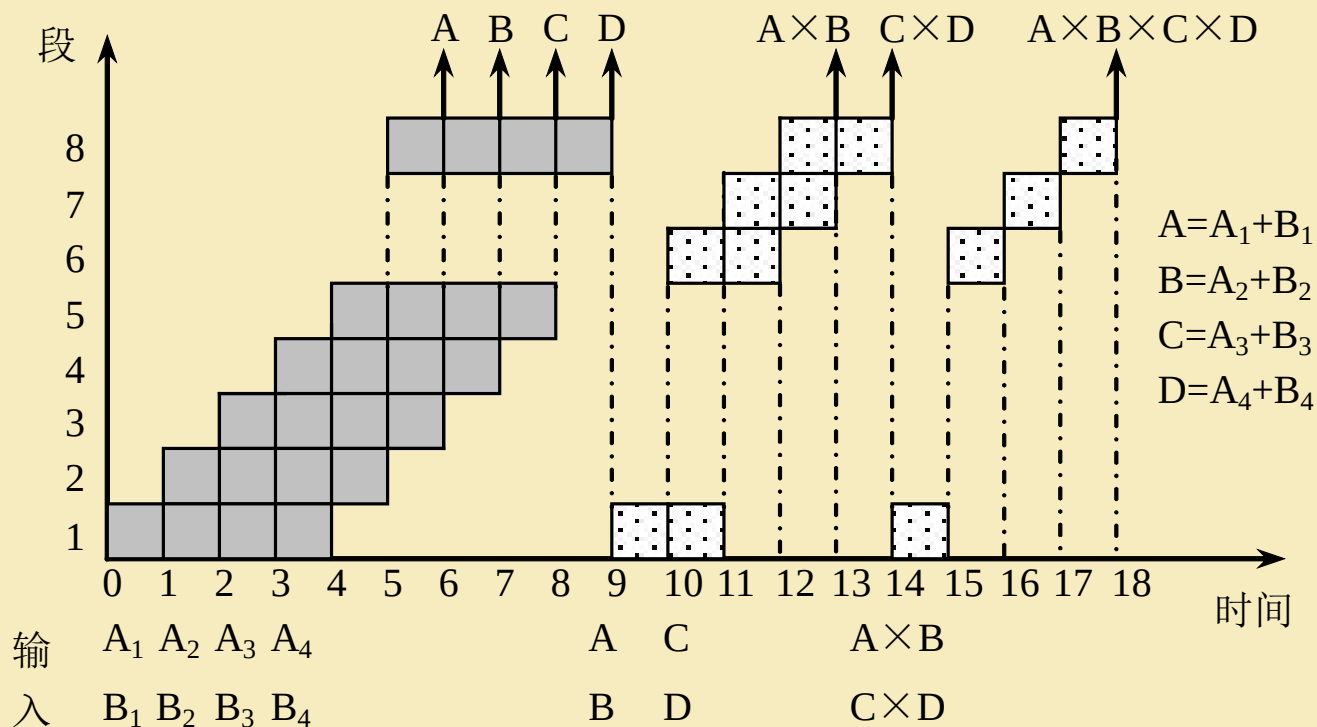
(每段的时间都为 Δt)

3.2 流水线的性能指标

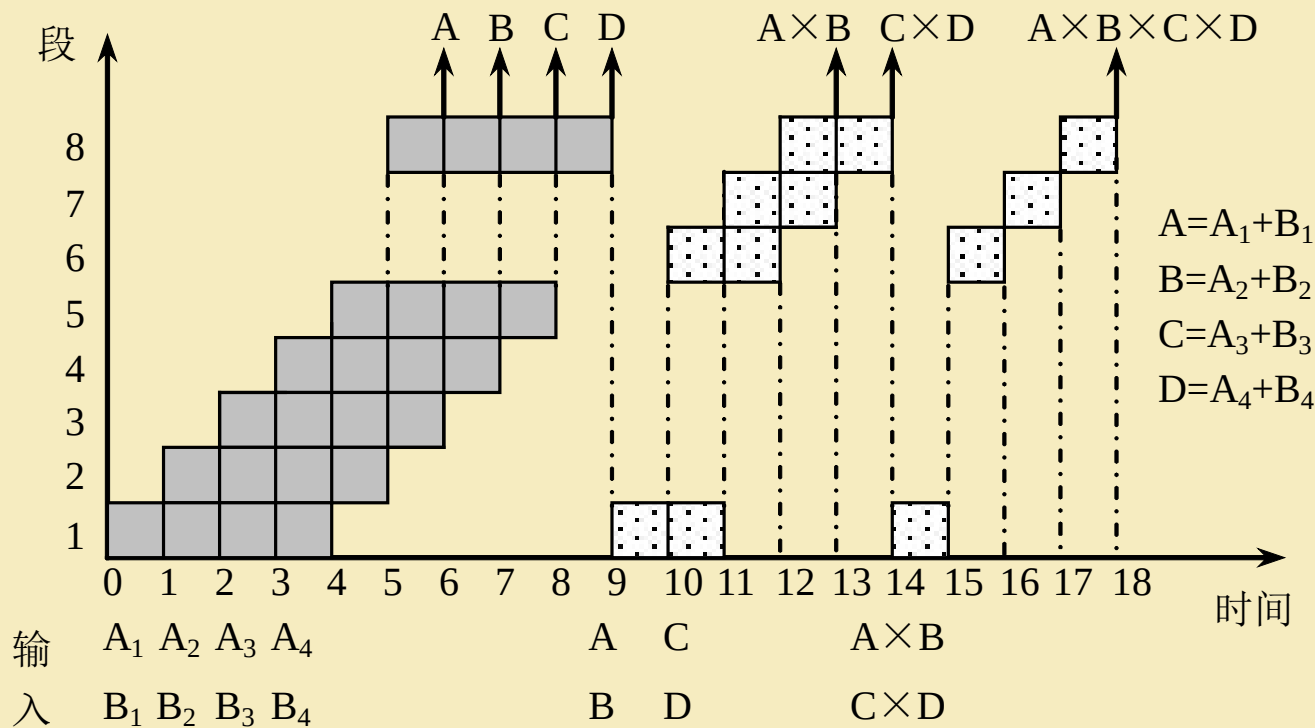
解： (1) 选择适合于流水线工作的算法

- 先计算 A_1+B_1 、 A_2+B_2 、 A_3+B_3 和 A_4+B_4 ；
- 再计算 $(A_1+B_1) \times (A_2+B_2)$ 和 $(A_3+B_3) \times (A_4+B_4)$ ；
- 然后求总的乘积结果。

(2) 画出时空图



3.2 流水线的性能指标



吞吐率：18个时间间隔，7个结果：
$$TP = \frac{n}{T_k} = \frac{7}{18\Delta t}$$

加速比：无流水线一次求和 $6\Delta t$ ，一次求积 $4\Delta t$ ，7个结果共需 $36\Delta t$ ：
$$S = \frac{36\Delta t}{18\Delta t} = 2$$

效率：阴影面积比8个段总时空区面积：
$$E = \frac{4 \times 6 + 3 \times 4}{8 \times 18} = 0.25$$

3.2 流水线的性能指标

- ▣ 流水线的效率 25%

可以看出，在求解此问题时，该流水线的效率不高。

➤ 主要原因

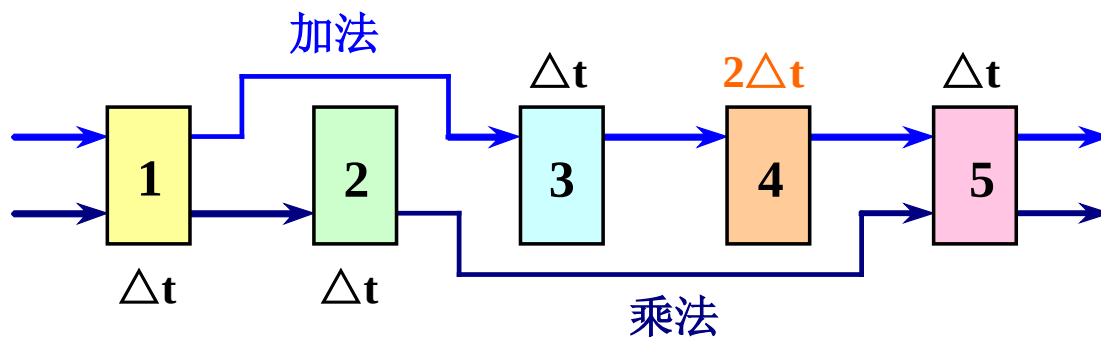
- ▣ 多功能流水线在做某一种运算时，总有一些段是空闲的；
- ▣ 静态流水线在进行功能切换时，要等前一种运算全部流出流水线后才能进行后面的运算；
- ▣ 运算之间存在关联，后面有些运算要用到前面运算的结果；
- ▣ 流水线的工作过程有建立与排空部分。

3.2 流水线的性能指标

例3.2 有一条动态多功能流水线由5段组成，加法用1、3、4、5段，乘法用1、2、5段，第4段的时间为 $2\Delta t$ ，其余各段时间均为 Δt ，而且流水线的输出可以直接返回输入端或暂存于相应的流水寄存器中。若在该流水线上计算：

$$\sum_{i=1}^4 (A_i \times B_i)$$

试计算其吞吐率、加速比和效率。



3.2 流水线的性能指标

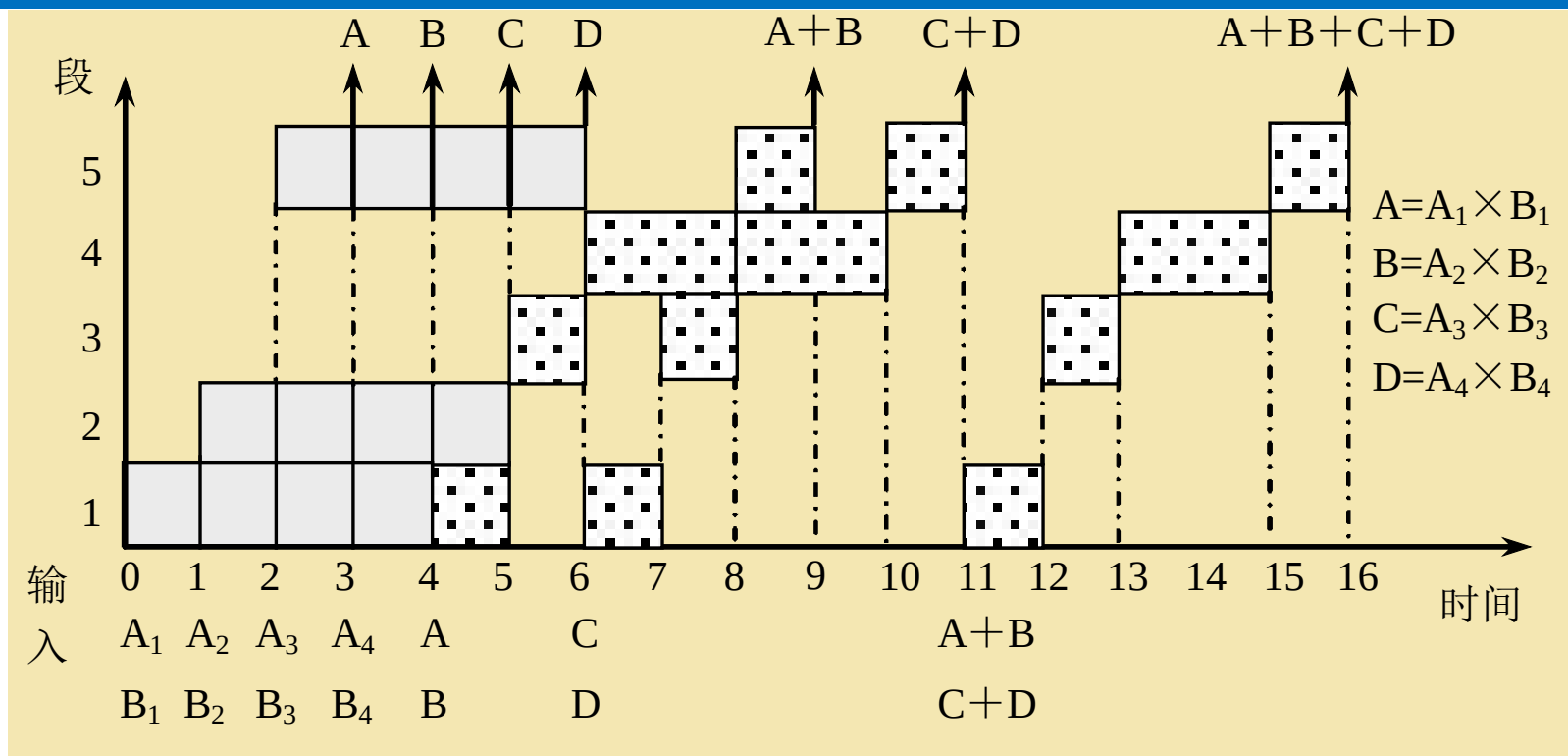
解：(1) 选择适合于流水线工作的算法

- 应先计算 $A_1 \times B_1$ 、 $A_2 \times B_2$ 、 $A_3 \times B_3$ 和 $A_4 \times B_4$ ；
- 再计算 $(A_1 \times B_1) + (A_2 \times B_2)$
 $(A_3 \times B_3) + (A_4 \times B_4)$ ；
- 然后求总的累加结果。

(2) 画出时空图

(3) 计算性能

3.2 流水线的性能指标



吞吐率：16个时间间隔，7个结果：
$$TP = \frac{7}{16\Delta t}$$

加速比：无流水线一次求积 $3\Delta t$ ，一次求和 $5\Delta t$ ，7个结果共需 $27\Delta t$ ：
$$S = \frac{27\Delta t}{16\Delta t} \approx 1.69$$

效率：阴影面积比5个段总时空区面积：
$$E = \frac{4 \times 3 + 3 \times 5}{5 \times 16} \approx 0.338$$

例：一个指令流水线有4个段，每个段的延迟时间分别为6ns，5ns，14ns和5ns。如果段间流水寄存器是1ns，求在稳定的理想状态下这个流水线与非流水线的加速比。

解：如果执行N条指令，不使用流水线时时间是
 $(6+5+14+5) * N = 30N$

如果采用流水线方式：

流水线周期： $\max\{6,5,14,5\} + 1 = 15$

当N足够大 加速比 $S = 30N / 15N = 2$

3.2 流水线的性能指标

3.2.5 流水线设计中的若干问题

1. 瓶颈问题

- 理想情况下，流水线在工作时，其中的任务是同步地每一个时钟周期往前流动一段。
- 当流水线各段不均匀时，机器的时钟周期取决于瓶颈段的延迟时间。
- 在设计流水线时，要尽可能使各段时间相等。

3.2 流水线的性能指标

2. 流水线的额外开销

- 流水寄存器需要建立时间和传输延迟
 - **建立时间**：在触发写操作的时钟信号到达之前，寄存器输入必须保持稳定的时间。
 - **传输延迟**：时钟信号到达后到寄存器输出可用的时间。
- 时钟偏移开销
 - 流水线中，时钟到达各流水寄存器的最大差值时间。（时钟到达各流水寄存器的时间不是完全相同）
- ◆ 流水线并不能减少（一般是增加）单条指令的执行时间，但却能提高吞吐率。
- ◆ 增加流水线的深度（段数）可以提高流水线的性能。
- ◆ 流水线的深度受限于流水线的额外开销。
- ◆ 当时钟周期小到与额外开销相近时，流水线已没意义。
===》流水线的段数愈多愈好？

3.2 流水线的性能指标

3. 冲突问题

后面计算要用到前面的计算结果；指令流水线中指令之间存在关联，则要相互等待引起流水线停顿。

-----流水线设计中要解决的**重要问题之一**。

- 在非线性流水线中，存在反馈回路，当一个任务在流水线中流过时，可能要多次经过某些段。
- 流水线调度要**解决的问题**：

应按什么样的时间间隔向流水线输入新任务，才能既不发生功能段使用冲突，又能使流水线有较高的吞吐率和效率？

3.4 相关与冲突

3.4 流水线的相关与冲突

3.4.1 一条经典的5段RISC流水线

首先讨论在非流水情况下是如何实现的

1. 一条指令的执行过程分为以下5个时钟周期：

➤ 取指令周期（IF）

- 以程序计数器PC中的内容作为地址，从存储器中取出指令并放入指令寄存器IR；
- 同时PC值加4（假设每条指令占4个字节），指向顺序的下一条指令。

➤ 指令译码/读寄存器周期（ID）

对指令进行译码，并用IR中的寄存器地址去访问通用寄存器组，读出所需的操作数。

3.4 流水线的相关与冲突

➤ 执行/有效地址计算周期（EX）

不同指令所进行的操作不同：

- ❑ **load和store指令**：ALU把指令中所指定的寄存器的内容与偏移量相加，形成访存有效地址。
- ❑ **寄存器—寄存器ALU指令**：ALU按照操作码指定的操作对从通用寄存器组中读出的数据进行运算。
- ❑ **寄存器—立即数ALU指令**：ALU按照操作码指定的操作对从通用寄存器组中读出的操作数和指令中给出的立即数进行运算。
- ❑ **分支指令**：ALU把指令中给出的偏移量与PC值相加，形成转移目标的地址；同时，对在前一个周期读出的操作数进行判断，确定分支是否成功。

3.4 流水线的相关与冲突

➤ 存储器访问 / 分支完成周期（MEM）

该周期处理的指令只有load、store和分支指令。
其它类型的指令在此周期不做任何操作。

□ load和store指令

load指令：用上一个周期计算出的有效地址从存储器中
读出相应的数据；

store指令：把指定的数据写入这个有效地址所指出的存
储器单元。

□ 分支指令

分支“成功”，就把转移目标地址送入PC。

分支指令执行完成。

3.4 流水线的相关与冲突

➤ 写回周期（WB）

ALU运算指令和load指令在这个周期把结果数据写入通用寄存器组。

ALU运算指令：结果数据来自ALU。

load指令：结果数据来自存储器。

在这个实现方案中：

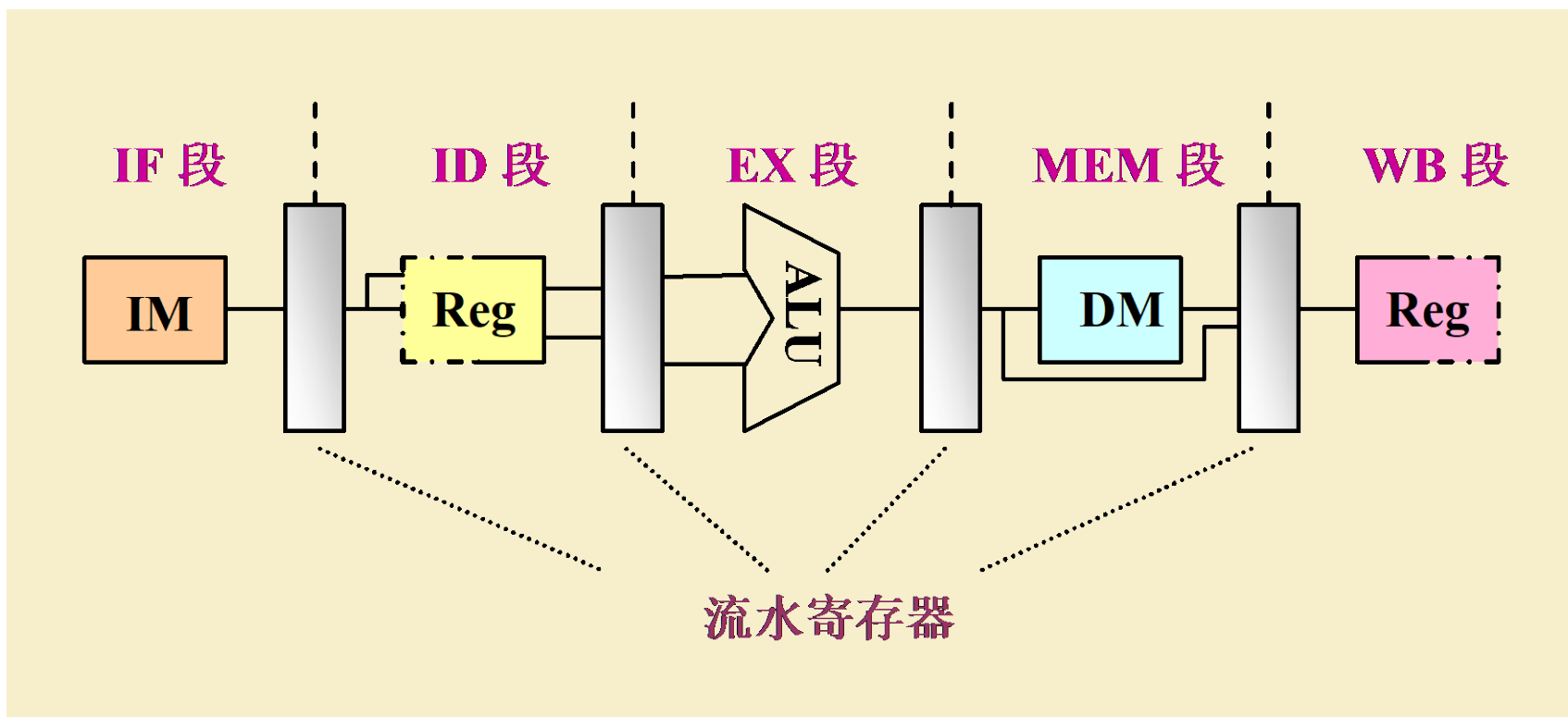
- store指令需要4个周期；
- 分支指令需要4个时钟周期
（如果把分支指令的执行提前到ID周期，则只需要2个周期）；
- 其它指令（一般情况）需要5个周期完成。

3.4 流水线的相关与冲突

2. 将上述实现方案修改为流水线实现

➤ 一条经典的5段流水线

- 每一个周期作为一个流水段；
- 在各段之间加上锁存器（流水寄存器）。



3.4 流水线的相关与冲突

3. 采用流水线实现时，应解决以下几个问题：

- 要保证不会在同一时钟周期要求同一个功能段做两件不同的工作。

例如：不能要求ALU同时做有效地址计算和算术运算。

- 避免IF段的访存（取指令）与MEM段的访存（读/写数据）发生冲突。
 - ❑ 采用分离的指令存储器和数据存储器；
 - ❑ 采用分离的指令Cache和数据Cache。
- ID段和WB段都要访问同一寄存器文件。

ID段：读

WB段：写

如何解决对同一寄存器的访问冲突？

把写操作安排在时钟周期的前半拍完成，

把读操作安排在后半拍完成。

3.4 流水线的相关与冲突

➤ 考虑PC的问题

- ❑ 流水线为了能够每个时钟周期启动一条新的指令，就必须在每个时钟周期进行PC值的加4操作，并保留新的PC值。这种操作**必须在IF段完成**，以便为取下一条指令做好准备。

（需设置一个专门的加法器）

- ❑ 但分支指令也可能改变PC的值，而且是在MEM段进行，这会导致冲突。

请考虑一下，如何处理分支指令？

3.4 流水线的相关与冲突

4. 5段流水线时空图的另两种描述方式

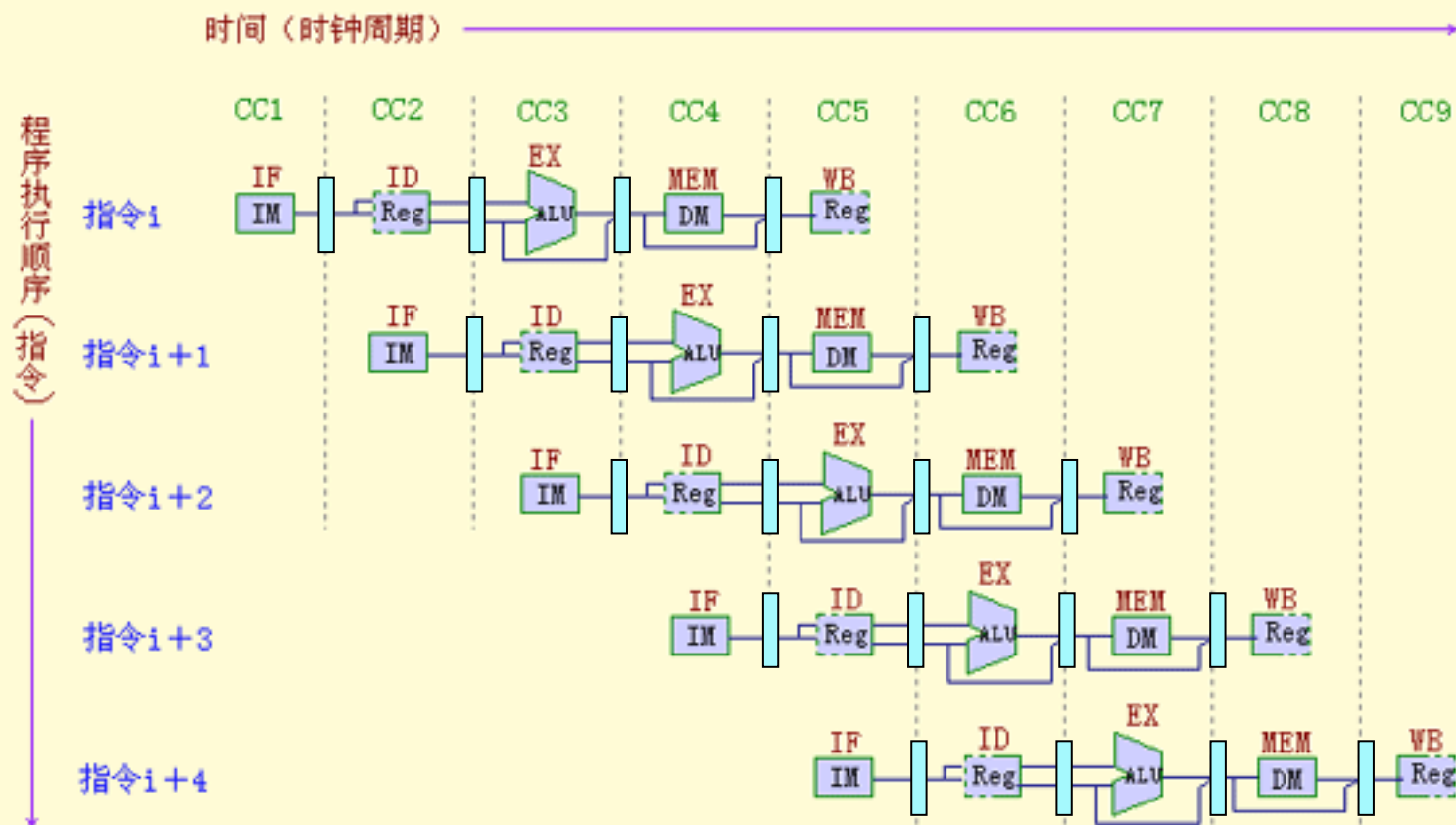
□ 第一种描述

	时钟周期								
指令编号	1	2	3	4	5	6	7	8	9
指令i	IF	ID	EX	MEM	WB				
指令i+1		IF	ID	EX	MEM	WB			
指令i+2			IF	ID	EX	MEM	WB		
指令i+3				IF	ID	EX	MEM	WB	
指令i+4					IF	ID	EX	MEM	WB

3.4 流水线的相关与冲突

➤ 第二种描述（按时间错开的数据通路序列）

流水线可以看成是按时间错开的数据通路序列



这种方式更直观地展现了部件重叠工作的情况。

3.4 流水线的相关与冲突

3.4.2 相关与流水线冲突

3.4.2.1 相关

- **相关(Dependence)**: 两条指令之间存在某种依赖关系。
如果两条指令相关, 则它们就有可能不能在流水线中重叠执行或者只能部分重叠执行。
- 相关有3种类型
 - ❑ 数据相关 (也称真数据相关, **Data Dependence**)
 - ❑ 名相关 (**Name Dependence**)
 - ❑ 控制相关 (**Control Dependence**)

3.4 流水线的相关与冲突

1. 数据相关

- 对于两条指令*i*（在前，下同）和*j*（在后，下同），如果下述条件之一成立，则称指令*j*与指令*i*数据相关。
 - 指令*j*使用指令*i*产生的结果；
 - 指令*j*与指令*k*数据相关，而指令*k*又与指令*i*数据相关。——数据相关具有传递性。

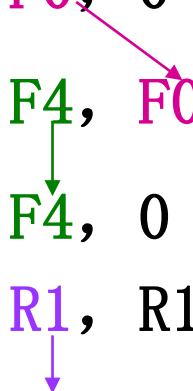
数据相关反映了数据的流动关系，

即如何从其产生者流动到其消费者。

3.4 流水线的相关与冲突

例如：下面这一段代码存在数据相关。

```
Loop:  L. D      F0, 0 (R1)      // F0为数组元素
        ADD. D   F4, F0, F2      // 加上F2中的值
        S. D     F4, 0 (R1)      // 保存结果
        DADDIU   R1, R1, -8      // 数组指针递减8个字节
        BNE      R1, R2, Loop    // 如果R1≠R2, 则分支
```



- 当数据的流动是经过寄存器时，相关的检测比较直观容易。
- 当数据的流动是经过存储器时，检测比较复杂。
 - 相同形式的地址其有效地址未必相同；
 - 形式不同的地址其有效地址却可能相同。

3.4 流水线的相关与冲突

2. 名相关

- **名**：指令所访问的寄存器或存储器单元的名称。
- 如果两条指令使用相同的名，但是它们之间并没有数据流动，则称这两条指令存在**名相关**。
- **指令j与指令i**之间的名相关有两种：
 - **反相关**：如果指令j写的名与指令i读的名相同，则称指令i和j发生了反相关。

指令j写的名=指令i读的名

- **输出相关**：如果指令j和指令i写相同的名，则称指令i和j发生了输出相关。

指令j写的名=指令i写的名

3.4 流水线的相关与冲突

- 名相关的两条指令之间并没有数据的传送。
 - 如果一条指令中的名改变了，并不影响另外一条指令的执行。
- 换名技术（Renaming）
- **换名技术**：通过改变指令中操作数的名来消除名相关。
 - 对于寄存器操作数进行换名称为**寄存器换名**。
- 既可以用编译器静态实现，也可以用硬件动态完成。

3.4 流水线的相关与冲突

例如：考虑下述代码：

DIV. D	F2, F8, F4
ADD. D	F8, F0, F12
SUB. D	F10, F8, F14

DIV. D和ADD. D存在反相关，

进行寄存器换名（F8换成S）后，变成：

DIV. D	F2, S, F4
ADD. D	S, F0, F12
SUB. D	F10, S, F14

3.4 流水线的相关与冲突

3. 控制相关

- **控制相关**是指由分支指令引起的相关。
 - ❑ 为了保证程序应有的执行顺序，必须严格按控制相关确定的顺序执行。
- 典型的程序结构是“if-then”结构：

```
if p1 {  
    S1;  
};  
S;  
if p2 {  
    S2;  
};
```

S1与p1控制相关，S2与p2控制相关，S与p1和p2均无关。

3.4 流水线的相关与冲突

➤ 控制相关带来了以下两个限制：

- 与一条分支指令控制相关的指令不能被移到该分支之前。否则这些指令就不受该分支控制了。

对于上述的例子，**then** 部分中的指令不能移到**if** 语句之前。

- 如果一条指令与某分支指令不存在控制相关，就不能把该指令移到该分支之后。

对于这个例子，不能把**S**移到**if**语句的**then** 部分中。

```
if p1 {  
    S1;  
};  
S;  
if p2 {  
    S2;  
};
```